



PROJECT-BASED LEARNING

PHASE-I EVALUATION

Smart Student Life: Campus Assistant

Team ID: DSCPP-III-2026-T307

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T307
Semester: 3rd
Section: ML1,ML3,ML4
Domain: Data Structures & OOPs

Team Members

1. Solanki, Anshuman – 2512510022
2. Mundepi, Arush – 2510370121
3. Anubhav Singh – 2510370452

Mentor

Dr. Jyoti Agarwal

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Students manage assignments, exams, attendance, library activities, college events and placement opportunities through separate sources. This makes it difficult to track pending work and important deadlines. The proposed system brings these activities together in one console-based application and prioritizes tasks according to their deadlines.

Motivation

- ▶ Reduce scattered academic and campus information.
- ▶ Help students track pending activities and deadlines.
- ▶ Show urgent tasks first using priority-based processing.

Target Users / Stakeholders

- ▶ College and university students.
- ▶ Students managing multiple academic tasks.
- ▶ Students tracking campus activities and deadlines.

Objectives

Project Objectives

1. Develop a console-based assistant for common academic and campus activities.
2. Organize student information through separate functional modules.
3. Compare task due dates with the current system date.
4. Calculate task urgency from due dates and display tasks in priority order using a Priority Queue.
5. Integrate Data Structures implemented in C with OOP concepts implemented in C++.
6. Store information permanently using file handling.

Key Objective: Help students organize college activities and focus first on tasks with the nearest deadlines.

Proposed Solution

Proposed Approach

1. Enter activity/task details and due date.
2. Validate and record the task information.
3. Compare current date with the due date.
4. Calculate days remaining and assign priority.
5. Priority Queue shows urgent tasks first.
6. File handling stores data for future access.

Key Features

- ▶ Student Profile
- ▶ Assignments and Exams
- ▶ Attendance Management
- ▶ Library Management
- ▶ College Events
- ▶ Placement Opportunities
- ▶ Deadline-based Priority System
- ▶ Add, update, complete and delete tasks

Priority Rule: P1 = overdue; P2 = today; P3 = within 3 days; P4 = within 7 days; P5 = later.

Scope: Manual/local data only; no live university-system integration.

Integration of PBL Course Concepts

Data Structures in C

Linked List	Dynamic task/activity records
Queue	Normal pending activities in order
Priority Queue	Tasks ordered by deadline urgency
Stack	Recent actions / undo history
BST	Search records by task ID or date

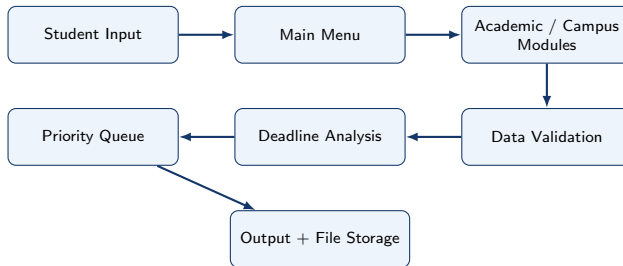
OOPs with C++

Classes / Objects	Student and activity objects
Encapsulation	Protect student/task data
Inheritance	Activity base class for Assignment/Event/Exam
Polymorphism	Common interface for activity types

File Handling: File read/write for persistent local student and task data. **Date Handling:** Current date vs. due date for days remaining and priority.

C-C++ Integration: C implements core data-structure modules; C++ manages student/activity classes and uses the C modules for required data processing.

System Architecture / Workflow



Major Components

- ▶ Student Profile and activity records
- ▶ Academic: Assignments, Exams, Attendance
- ▶ Campus: Library, Events, Placements
- ▶ Deadline-based Priority System

Data / Control Flow

- ▶ Enter and validate activity/due-date data
- ▶ Compare due date with current date
- ▶ Calculate days remaining and priority
- ▶ Display result and save data to files

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C and C++
- ▶ Interface: Console-based application
- ▶ Storage: Local files using file handling
- ▶ Libraries: Standard C/C++ libraries
- ▶ IDE: VS Code / Code::Blocks

Development Methodology

1. Requirement Analysis
2. System Design
3. Data Structure and OOP Design
4. Module Implementation
5. Integration and Testing
6. Final Evaluation

C/C++ are selected to demonstrate the required Data Structures and OOP concepts in the same project.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Problem statement and project scope finalized.
- ▶ Main academic and campus modules identified.
- ▶ Deadline-based priority workflow designed.
- ▶ Required Data Structures identified.
- ▶ OOP and file-handling approach planned.
- ▶ Initial console workflow and module structure prepared.

Individual Contribution

Member	Role	Contribution
Solanki, Anshuman	Team Lead	Project planning, architecture, priority system and integration.
Mundepi, Arush	Developer	Data Structures, OOP implementation and module development.
Anubhav Singh	Testing / Documentation	File handling, testing and project documentation.

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal and System Design
2. Phase-II: Development, Data Structure/OOP Implementation and Integration
3. Phase-III: Functional Testing, Performance Validation and Final Implementation

Expected Outcomes

- ▶ Working console-based student assistant.
- ▶ Integrated academic and campus modules.
- ▶ Functional deadline-based priority mechanism.
- ▶ Persistent local data storage.
- ▶ Tested task lifecycle: add, update, complete and delete.

References

1. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*.
2. E. Balagurusamy, *Object-Oriented Programming with C++*.
3. Data Structures and Algorithms course material, Graphic Era (Deemed to be University).
4. *cppreference*, C and C++ language / standard library documentation.

Scope / Limitations

- ▶ Console-based interface with manual data entry.
- ▶ Local file storage; no live university portal/API integration.

Thank You
Questions & Suggestions